AGRIFLEX

Exigences fonctionnelles et non fonctionnelles
Architecture de l'application et intégration de la base de données

Projet académique : Solution de recrutement intégrée – Groupe KIBA

Document de conception — Proposition élaborée à partir du document de cadrage KIBA

1. Analyse des exigences de l'application AgriFlex

1.1. Présentation générale

AgriFlex est une application numérique de recrutement temporaire destinée au Groupe KIBA. Elle vise à faciliter la mise en relation entre les travailleurs non qualifiés, notamment les ouvriers agricoles et les manutentionnaires, et les entreprises ayant des besoins urgents en main-d'œuvre.

L'application doit permettre de simplifier l'inscription des candidats, de rechercher des travailleurs selon leur localisation et leur disponibilité, de gérer les missions temporaires et de faciliter le travail des agents de recrutement de KIBA.

Elle répond principalement à deux problèmes : la difficulté de recruter rapidement des travailleurs peu familiarisés avec les outils numériques, et la difficulté de gérer la mobilité géographique et la rotation saisonnière des travailleurs au Maroc.

1.2. Les acteurs de l'application

Acteur	Rôle et responsabilités
Candidat / travailleur	S'inscrire facilement ; renseigner ses informations personnelles, sa région, ses compétences et ses disponibilités ; recevoir des propositions de missions ; accepter ou refuser une mission.
Recruteur KIBA	Gérer les candidats ; consulter les disponibilités ; créer et gérer les offres de missions ; rechercher les profils adaptés ; suivre les affectations et les effectifs par région.
Administrateur (proposition)	Gérer les comptes des recruteurs ; contrôler les droits d'accès ; superviser les paramètres et les données de l'application.

Les candidats et les recruteurs KIBA sont explicitement identifiés dans le document de cadrage. Le rôle d'administrateur est une proposition d'organisation technique.

1.3. Les exigences fonctionnelles (EF)

Les exigences fonctionnelles décrivent ce que l'application doit faire et les services qu'elle doit fournir à ses utilisateurs.

A. Gestion des comptes et inscription des candidats

Référence	Exigence fonctionnelle
EF01	Permettre aux candidats de s'inscrire facilement sur l'application.
EF02	Permettre la saisie des informations essentielles : nom, prénom, téléphone et région.
EF03	Permettre au candidat de renseigner ses compétences et ses expériences éventuelles.
EF04	Permettre au candidat d'indiquer ses disponibilités et sa mobilité géographique.
EF05	Permettre la consultation et la modification de son profil.
EF06	Permettre une inscription simplifiée, sans obligation de créer un CV complexe.
EF07	Prévoir une interface simplifiée adaptée aux personnes peu familiarisées avec le numérique.

B. Authentification et gestion des utilisateurs

Référence	Exigence fonctionnelle
EF08	Permettre aux utilisateurs autorisés de se connecter à l'application.
EF09	Permettre la déconnexion sécurisée des utilisateurs.
EF10	Distinguer les droits des candidats et des recruteurs KIBA.
EF11	Permettre aux recruteurs de consulter les informations nécessaires à leur travail.
EF12	Permettre la gestion des comptes des recruteurs par un administrateur autorisé.

C. Gestion des offres de missions temporaires

Référence	Exigence fonctionnelle
EF13	Permettre aux recruteurs de créer une offre de mission.
EF14	Enregistrer le type de travail, la région, le nombre de travailleurs recherchés et les dates de la mission.
EF15	Permettre de consulter les missions disponibles.
EF16	Permettre de modifier ou d'annuler une mission.
EF17	Gérer le statut d'une mission : ouverte, en cours, terminée ou annulée.
EF18	Permettre aux recruteurs de suivre les besoins en main-d'œuvre des entreprises clientes.

D. Recherche et matching géographique

Référence	Exigence fonctionnelle
EF19	Rechercher des candidats selon leur région.
EF20	Filtrer les candidats selon leurs compétences et leurs disponibilités.
EF21	Comparer les besoins d'une mission avec les profils des travailleurs.
EF22	Proposer les travailleurs les plus adaptés à une mission.
EF23	Prendre en compte la mobilité des travailleurs entre les régions.
EF24	Éviter, autant que possible, de proposer une mission à un travailleur déjà affecté à une mission incompatible.

E. Gestion des candidatures et des affectations

Référence	Exigence fonctionnelle
EF25	Permettre au candidat de manifester son intérêt pour une mission.
EF26	Permettre au candidat d'accepter ou de refuser une proposition de mission.
EF27	Permettre au recruteur de consulter les réponses des candidats.
EF28	Permettre au recruteur d'affecter un candidat à une mission.
EF29	Enregistrer le statut de chaque affectation : proposée, acceptée, refusée ou terminée.
EF30	Suivre le nombre de travailleurs affectés et le nombre de postes restant à pourvoir.

F. Notifications

Référence	Exigence fonctionnelle
EF31	Envoyer une notification lorsqu'une mission adaptée est disponible.
EF32	Informer le candidat des détails essentiels de la mission.
EF33	Envoyer des notifications par SMS.
EF34	Informer le candidat lorsque le statut de sa candidature change.
EF35	Permettre de suivre l'état d'envoi des notifications.

G. Planification et gestion de la rotation saisonnière

Référence	Exigence fonctionnelle
EF36	Afficher les effectifs disponibles par région.
EF37	Visualiser les missions en fonction des dates et des régions.
EF38	Anticiper les besoins de main-d'œuvre pendant les périodes agricoles saisonnières.
EF39	Suivre les déplacements et les changements d'affectation des travailleurs.
EF40	Identifier les régions où les besoins en main-d'œuvre sont importants.

H. Tableau de bord et statistiques

Référence	Exigence fonctionnelle
EF41	Afficher le nombre total de candidats inscrits.
EF42	Afficher les missions ouvertes, en cours et terminées.
EF43	Afficher le nombre de postes à pourvoir et de travailleurs affectés.
EF44	Calculer le délai moyen de recrutement.
EF45	Suivre le taux d'acceptation des missions successives.
EF46	Suivre l'évolution des inscriptions et des besoins par région.

Précisions sur le périmètre fonctionnel

Le matching géographique, la prise en compte des disponibilités, le tableau de bord des effectifs par région et les alertes SMS sont directement liés à la vision du projet. Les autres détails de fonctionnement présentés dans cette liste constituent des propositions de conception à valider par l'équipe.

L'utilisation d'icônes, d'une interface vocale ou d'une intégration WhatsApp est une piste mentionnée dans le document de cadrage. La réalisation de chacune de ces options doit être confirmée par l'équipe.

1.4. Les exigences non fonctionnelles (ENF)

Les exigences non fonctionnelles décrivent comment l'application doit fonctionner, notamment en matière de qualité, de sécurité, de performance et d'accessibilité. Les valeurs chiffrées suivantes sont des objectifs proposés et non des engagements fixés par le document original.

Réf.	Catégorie	Exigence non fonctionnelle
ENF01	Ergonomie	L'interface doit être simple, intuitive et compréhensible par les travailleurs peu familiarisés avec les outils numériques.

Réf.	Catégorie	Exigence non fonctionnelle
ENF02	Accessibilité	Les boutons, les icônes et les formulaires doivent être facilement identifiables et utilisables sur un téléphone.
ENF03	Simplicité	L'inscription doit demander uniquement les informations nécessaires au recrutement.
ENF04	Performance	Les pages principales devraient s'afficher en moins de 3 secondes dans des conditions normales de connexion.
ENF05	Réactivité	La recherche et le filtrage des candidats devraient fournir leurs résultats en moins de 2 secondes pour une charge normale.
ENF06	Disponibilité	L'application devrait viser une disponibilité mensuelle d'au moins 99,5 %, hors maintenance planifiée.
ENF07	Sécurité	Les accès doivent être protégés par une authentification appropriée.
ENF08	Confidentialité	Les données personnelles des candidats ne doivent être accessibles qu'aux utilisateurs autorisés.
ENF09	Autorisations	Chaque rôle doit disposer uniquement des fonctionnalités et données nécessaires à ses responsabilités.
ENF10	Intégrité	Les informations relatives aux candidats, missions et affectations doivent rester cohérentes en base de données.
ENF11	Fiabilité	L'application doit gérer correctement les erreurs et éviter les doublons d'affectation incompatibles.
ENF12	Scalabilité	L'application doit pouvoir supporter une augmentation du nombre de candidats et de missions pendant les saisons agricoles.
ENF13	Compatibilité	L'interface candidat doit fonctionner sur les principaux navigateurs mobiles et ordinateurs ciblés.
ENF14	Connectivité	Les opérations essentielles doivent rester légères afin de limiter les difficultés sur les connexions Internet lentes.
ENF15	Maintenabilité	Le code doit être structuré en modules pour faciliter les corrections et les évolutions.
ENF16	Sauvegarde	La base de données doit faire l'objet de sauvegardes régulières et d'une procédure de restauration testée.
ENF17	Traçabilité	Les opérations importantes des recruteurs doivent pouvoir être retracées.
ENF18	Notifications	L'échec d'un envoi SMS doit être détectable et, si nécessaire, permettre une nouvelle tentative.
ENF19	Protection des données	La collecte et le traitement des données doivent respecter la réglementation applicable au Maroc.
ENF20	Robustesse	L'application doit rester stable lorsque plusieurs utilisateurs effectuent des opérations simultanément.

1.5. Priorisation des exigences

Priorité	Fonctionnalités
Priorité 1 — Essentielles	Inscription des candidats, connexion, gestion des profils, création des missions, recherche selon la région et les disponibilités, affectations et tableau de bord de base.
Priorité 2 — Importantes	Notifications SMS, matching plus avancé, suivi des rotations saisonnières et statistiques de recrutement.
Priorité 3 — Évolutions possibles	Inscription vocale, intégration WhatsApp, optimisation des déplacements entre régions et analyses prévisionnelles avancées.

Cette priorisation est une proposition pour organiser le développement ; elle ne constitue pas un planning officiellement imposé dans le document de cadrage.

2. Planification de l'architecture de l'application AgriFlex

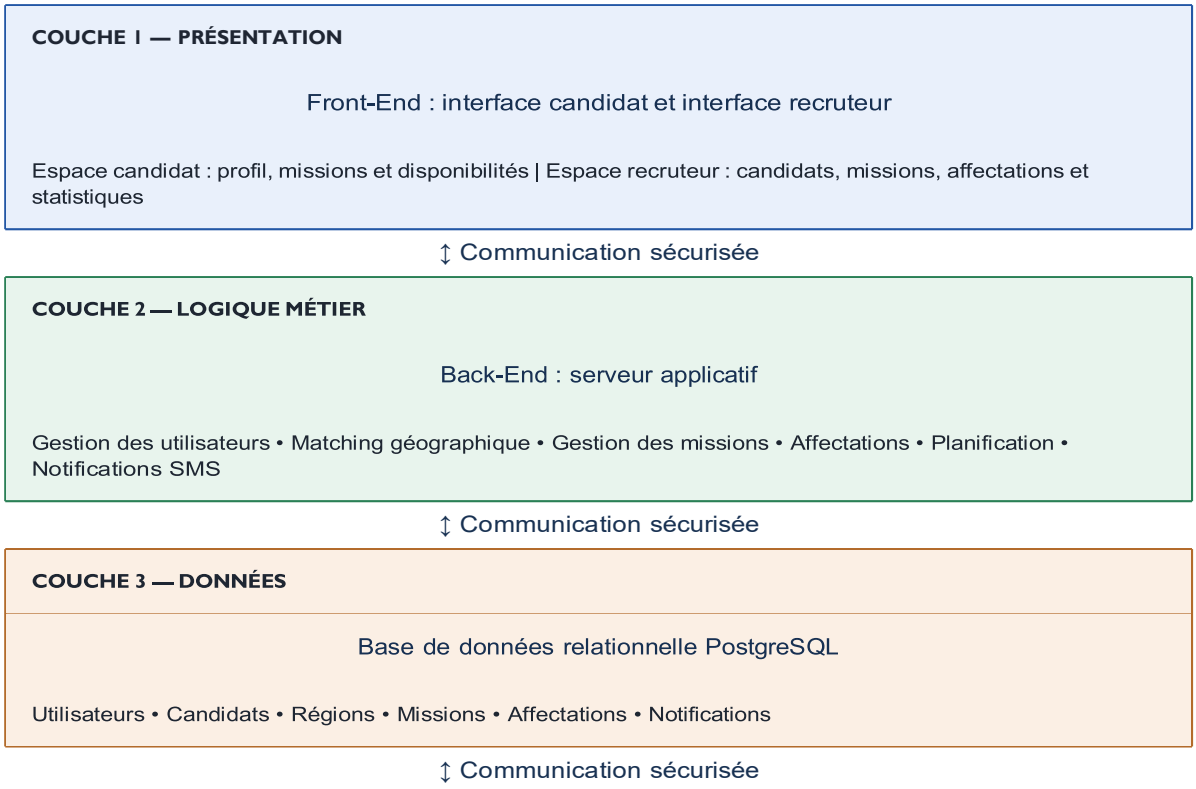
2.1. Objectif de l'architecture

L'architecture d'AgriFlex doit organiser les composants de l'application afin de faciliter la communication entre les candidats, les recruteurs KIBA et la base de données.

- Centraliser les informations des candidats et des missions.
- Automatiser la recherche des travailleurs selon les besoins des entreprises.
- Faciliter la gestion des affectations et de la rotation saisonnière.
- Sécuriser l'accès aux données.
- Assurer l'évolution de l'application et sa stabilité pendant les périodes de forte activité.

2.2. Architecture générale proposée

Pour ce projet académique, une architecture en trois couches (Three-Tier Architecture) est recommandée. Elle sépare l'interface utilisateur, la logique métier et la base de données.



Les échanges entre les couches passent par le serveur applicatif et des interfaces sécurisées.

Description des trois couches

Couche 1 — Présentation (Front-End). Elle représente les interfaces utilisées par les utilisateurs. L'interface candidat doit être simple et adaptée aux téléphones. L'interface recruteur doit proposer un tableau de bord, des filtres et des vues de planification. Les formulaires permettent d'envoyer des informations au serveur et d'afficher les résultats.

Couche 2 — Logique métier (Back-End). Elle traite les demandes des utilisateurs et applique les règles de gestion : vérifier les informations reçues, contrôler les droits d'accès, rechercher les candidats compatibles avec une mission, gérer les candidatures et affectations, calculer les statistiques et déclencher les notifications SMS.

Couche 3 — Base de données. Elle conserve les informations nécessaires au fonctionnement de l'application : utilisateurs, profils, missions, disponibilités, affectations et notifications.

2.3. Technologies proposées

Composant	Technologie proposée	Rôle
Interface	HTML, CSS, JavaScript	Construire les interfaces utilisateur.
Back-End	Java avec Spring Boot	Implémenter la logique métier et les API.
Base de données	PostgreSQL	Stocker et organiser les données.
Communication	API REST avec JSON	Échanger les données entre le Front-End et le Back-End.
Notifications	Service SMS via API	Envoyer les alertes aux candidats.
Conception	UML et Figma	Modéliser le système et concevoir les maquettes.
Gestion du code	Git et GitHub	Collaborer et versionner le code source.

Ces technologies sont une proposition cohérente pour le projet Java, et non des choix imposés par le document de cadrage. L'équipe peut les adapter à ses contraintes pédagogiques.

2.4. Conception de la base de données

La base de données doit être organisée en tables relationnelles reliées entre elles par des clés primaires (PRIMARY KEY) et des clés étrangères (FOREIGN KEY). Le modèle ci-dessous est une première proposition logique.

Table	Attributs principaux	Fonction
UTILISATEUR	id_utilisateur (PK) ; nom ; prénom ; téléphone ; mot_de_passe_hashé ; rôle	Stocker les comptes, les identifiants et les rôles.
CANDIDAT	id_candidat (PK) ; id_utilisateur (FK) ; id_region (FK) ; compétences ; disponibilité ; mobilité	Stocker les profils des travailleurs et leurs disponibilités.
REGION	id_region (PK) ; nom_region	Centraliser les régions pour faciliter le matching géographique.
MISSION	id_mission (PK) ; titre ; description ; type_travail ; id_region (FK) ; dates ; nombre_travailleurs ; statut ; id_recruteur (FK)	Enregistrer les offres de travail temporaire et les besoins en effectifs.
AFFECTATION	id_affectation (PK) ; id_candidat (FK) ; id_mission (FK) ; date_proposition ; statut	Relier les candidats aux missions et suivre leurs réponses.
NOTIFICATION	id_notification (PK) ; id_candidat (FK) ; id_mission (FK, facultatif) ; type ; contenu ; date_envoi ; statut_envoi	Conserver l'historique des alertes et l'état des envois.

Les tables CANDIDAT et UTILISATEUR sont séparées pour distinguer les informations de connexion des informations professionnelles. Les compétences peuvent être normalisées dans des tables spécifiques si plusieurs compétences doivent être associées à un même candidat.

Relation essentielle

Un candidat peut être associé à plusieurs missions et une mission peut concerner plusieurs candidats. Cette relation plusieurs-à-plusieurs est représentée par la table AFFECTATION.

2.5. Intégration entre l'application et la base de données

L'intégration suit un chemin clair : l'utilisateur réalise une action, le serveur vérifie les informations, puis il communique avec PostgreSQL.

Étape	Description
1. Action de l'utilisateur	Exemple : le recruteur crée une mission.
2. Requête Front-End	Le formulaire envoie les données à l'API REST.
3. Traitement Back-End	Validation, authentification et application des règles métier.
4. Accès à PostgreSQL	Enregistrement ou lecture des données.
5. Réponse du serveur	Le résultat est transmis à l'interface et affiché à l'utilisateur.

Exemple concret : recherche de 20 ouvriers agricoles

- Le recruteur saisit la région, la période et le nombre de travailleurs recherchés.
- Le Front-End envoie les critères au Back-End.
- Le Back-End recherche les candidats disponibles et vérifie leur compatibilité avec les critères de la mission.
- PostgreSQL fournit les profils correspondants.
- Le Back-End renvoie les résultats au recruteur.
- Le recruteur sélectionne les candidats et crée les affectations.
- Le système peut ensuite déclencher l'envoi de SMS aux travailleurs concernés.

Ce fonctionnement répond aux besoins de matching géographique, de recrutement rapide et de gestion de la rotation saisonnière décrits dans le document de cadrage.

2.6. Sécurité et fiabilité de l'intégration

Authentification : Stocker les mots de passe sous forme de hachages sécurisés, jamais en clair.

Contrôle des accès : Empêcher un candidat d'accéder aux fonctions réservées aux recruteurs.

Protection SQL : Utiliser des requêtes paramétrées ou un ORM comme JPA/Hibernate.

Validation : Vérifier les données côté serveur avant leur enregistrement.

Intégrité référentielle : Utiliser des clés étrangères pour empêcher les références invalides.

Gestion des transactions : Enregistrer les opérations liées ensemble de manière cohérente.

Sauvegarde : Prévoir des sauvegardes régulières et vérifier que la restauration fonctionne.

Protection des données personnelles : Limiter l'accès aux informations des candidats et respecter les obligations légales applicables.

2.7. Évolutivité de l'architecture

L'architecture devra pouvoir évoluer en fonction des besoins futurs du Groupe KIBA. Les évolutions possibles comprennent l'ajout d'une intégration WhatsApp, l'amélioration de l'algorithme de matching, l'ajout d'indicateurs au tableau de bord, l'augmentation des ressources serveur lors des pics saisonniers et l'ajout de fonctionnalités sans réécriture complète de l'application.

Pour une première version académique, une architecture en trois couches avec un Back-End Java/Spring Boot et une base PostgreSQL est un choix raisonnable. Une architecture distribuée en microservices ne paraît pas nécessaire au départ, sauf contrainte particulière.

Conclusion

L’analyse des exigences fonctionnelles et non fonctionnelles d’AgriFlex permet de définir les fonctionnalités essentielles et les contraintes de qualité de l’application. Les exigences fonctionnelles couvrent principalement l’inscription simplifiée des candidats, la gestion des missions temporaires, le matching géographique, les notifications SMS, la planification des effectifs et le suivi des indicateurs de recrutement.

Les exigences non fonctionnelles concernent notamment l’ergonomie, l’accessibilité, la performance, la sécurité, la confidentialité, la fiabilité et l’évolutivité du système.

Pour répondre à ces besoins, une architecture en trois couches est proposée : une couche de présentation pour les interfaces, une couche métier pour le traitement des opérations et une couche de données reposant sur une base de données relationnelle PostgreSQL. La communication entre les composants s’effectuera par l’intermédiaire d’API sécurisées.

Cette architecture constitue une base pour la conception technique d’AgriFlex. Elle devra être complétée par un diagramme de classes UML détaillé, un schéma relationnel définitif et des maquettes d’interface cohérentes avec les exigences retenues par l’équipe.